

Furniture E-Commerce

Complete Architecture, Data Schemas, APIs & Operational Specifications

React 19 & SSR

Node.js / Express 5

PostgreSQL DB

Stripe Checkout

Cloudinary Media

Redux Toolkit

System Overview & Documentation Purpose

This document serves as the comprehensive technical reference manual for the Furniture E-Commerce application. It covers full end-to-end architecture, multi-role use cases, detailed PostgreSQL schemas across all 9 relational entities, interactive flowcharts, full REST API specifications, security mechanisms, directory organization, and deployment workflows.

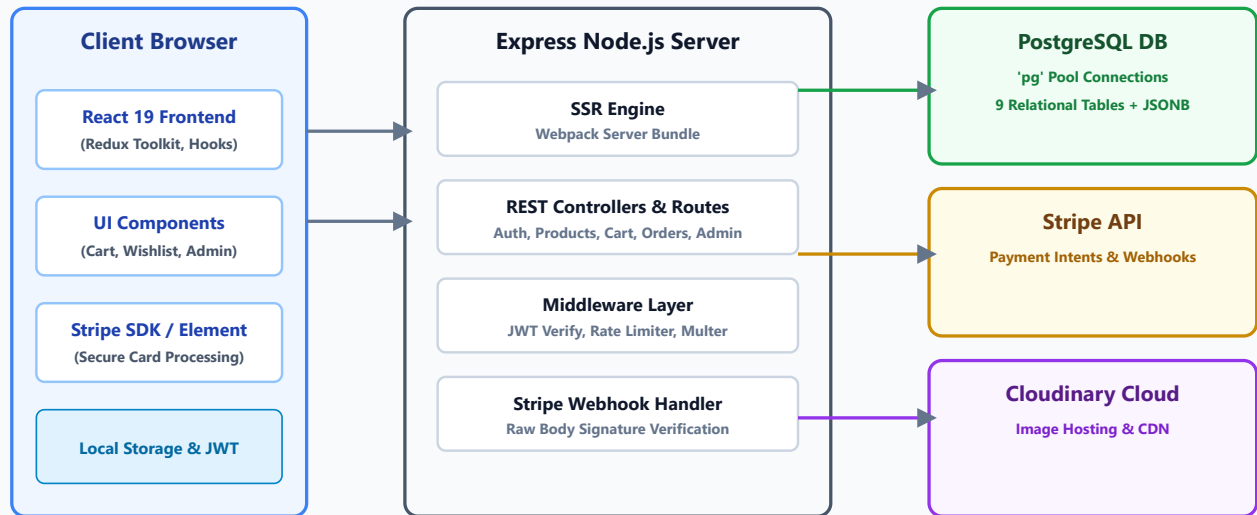
Core Technology Specifications

- **Frontend Stack:** React 19, React Router 7, Redux Toolkit, Lucide Icons, Webpack Bundler
- **Server-Side Rendering (SSR):** Custom Webpack SSR pipeline with HTML string stream injection
- **Backend Engine:** Node.js with Express 5, JWT Authentication & Bcrypt Hashing
- **Database Layer:** PostgreSQL pool connections with JSONB dynamic attributes
- **External Services:** Stripe API Payment Gateway & Webhook Signature Verification, Cloudinary Asset Hosting

1. System Architecture & Overview

The system is built as a high-performance e-commerce platform combining initial Server-Side Rendering (SSR) for fast first-page load and search engine indexing with dynamic React single-page application (SPA) interactivity for seamless browsing, cart operations, and administrative dashboards.

FIGURE 1.1: END-TO-END SYSTEM ARCHITECTURE OVERVIEW



2. System Use Cases & Role Matrix

The application defines clear operational privileges across three user roles: Guests, Authenticated Customers, and System Administrators.

2.1 Guest (Unauthenticated User) Capabilities

- **Catalog Browsing:** Explore furniture categories, filter products by category, price, or keywords, and view detailed product specs.
- **Interactive Gallery:** Scroll through image carousels powered by Embla / Keen-Slider.
- **Reviews Inspection:** Read customer product reviews and overall ratings without login requirement.
- **User Registration:** Create a new customer account using full name, valid email, and secure password.

2.2 Authenticated Customer Capabilities

- **Cart Session Management:** Add products to cart, increment/decrement item quantities, and remove items with real-time total price calculation.
- **Wishlist Operations:** Bookmark products to personal wishlist for future sessions.
- **Stripe Checkout Integration:** Initiate secure payments via Stripe Checkout sessions, compute dynamic shipping rates, and complete transactions.
- **Order History & Tracking:** Access historical order list, view itemized breakdown, and check status (pending , paid , shipped , completed).
- **Profile Customization:** Update profile info, contact numbers, shipping addresses, and upload personal avatar images directly to Cloudinary.
- **Product Reviews:** Submit star ratings and text reviews for purchased products.

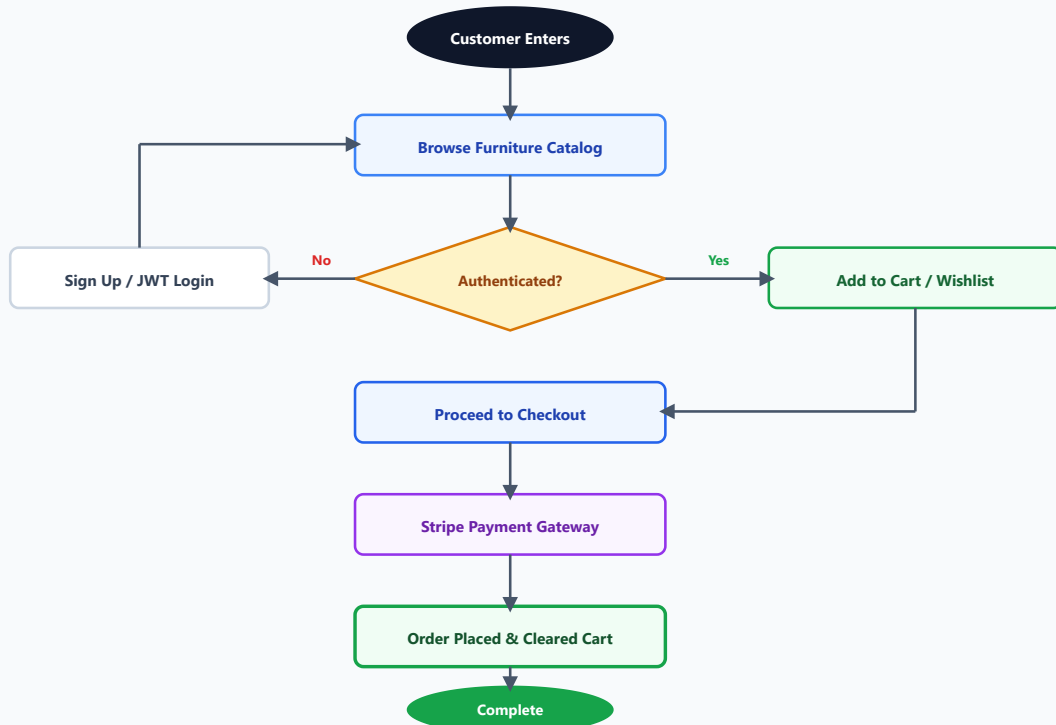
2.3 Administrator Capabilities

- **Analytics Dashboard:** View system statistics including total sales revenue, order volumes, customer registrations, and inventory counts.
- **Product & Category Management (CRUD):**** Add new furniture items, edit prices/descriptions/stock, delete items, and create categories with Cloudinary image uploads.
- **Order Fulfillment Controls:** Inspect all store orders, view user details, and update fulfillment statuses.
- **Customer Management:** View user accounts and contact details.
- **Dynamic Store Settings (JSONB):** Update store information (name, email, phone, currency), toggle Stripe test mode, configure flat-rate shipping or free-shipping thresholds, and manage Cloudinary presets.

3. System Workflows & Flowcharts

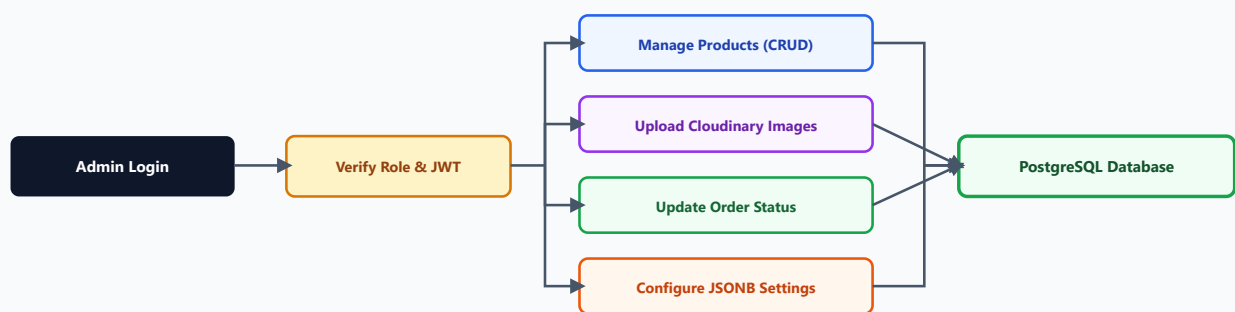
3.1 Customer Journey & Checkout Flow

FIGURE 3.1: CUSTOMER SHOPPING & CHECKOUT FLOWCHART



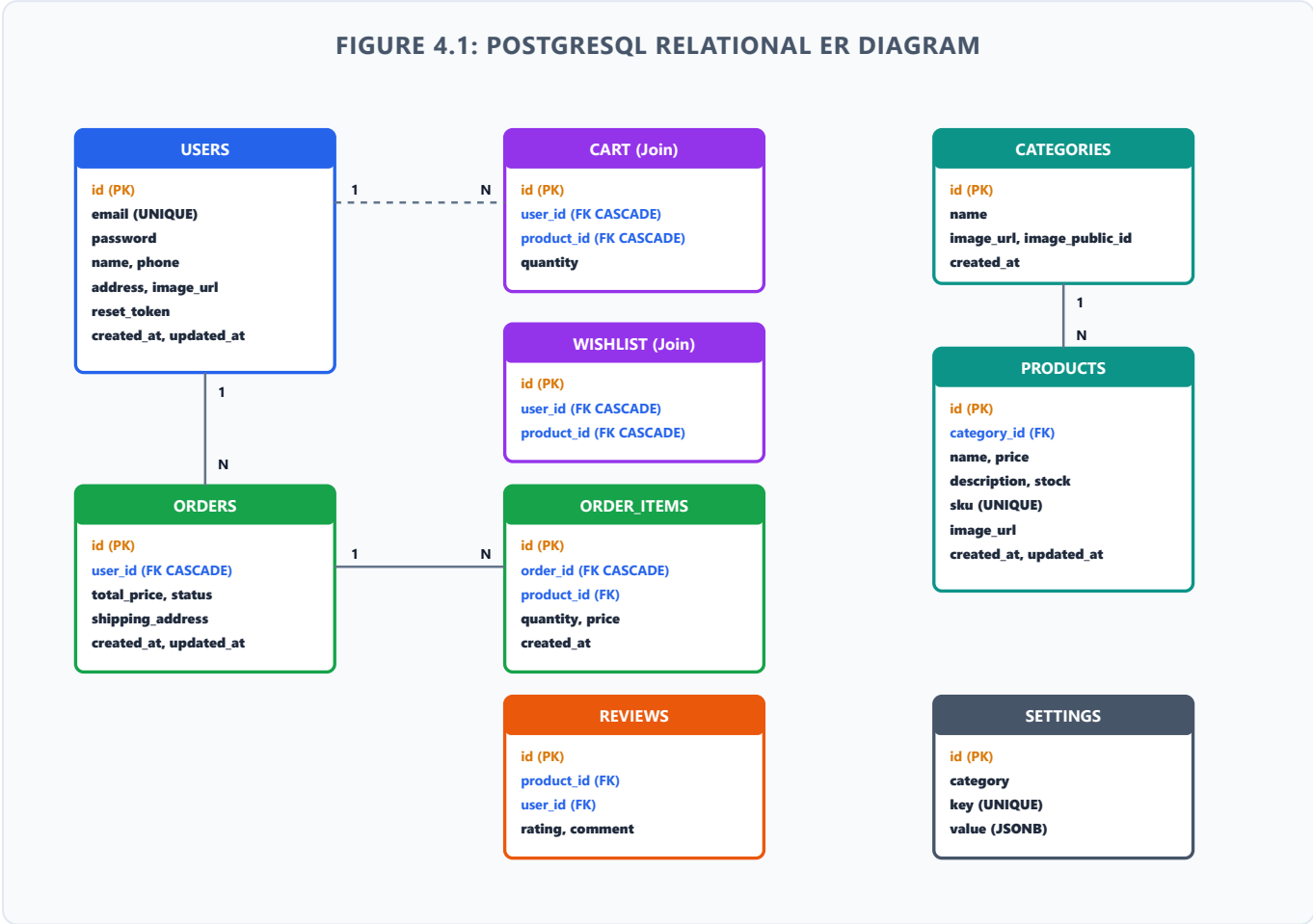
3.2 Admin Catalog & Store Management Flowchart

FIGURE 3.2: ADMIN WORKFLOW DIAGRAM



4. Database ER Diagram & Relational Schemas

The system utilizes PostgreSQL with 9 relational tables enforcing foreign key constraints, unique indexes, cascading deletes, and JSONB dynamic key-value metadata.



4.1 Table: users

Column	Data Type	Constraints & Defaults	Description
id	SERIAL	PRIMARY KEY	Unique auto-incrementing user ID
email	VARCHAR(255)	NOT NULL, UNIQUE	User login email address
password	VARCHAR(255)	NOT NULL	Bcrypt hashed user password
name	VARCHAR(255)	NOT NULL	Full user profile name
phone	VARCHAR(20)	NULL	Contact telephone number
address	TEXT	NULL	Physical shipping address
image_url	TEXT	NULL	Cloudinary avatar image URL
image_public_id	VARCHAR(255)	NULL	Cloudinary asset public reference ID
reset_token	TEXT	NULL	Password reset security token

Column	Data Type	Constraints & Defaults	Description
reset_token_expiry	TIMESTAMP	NULL	Reset token expiration timestamp
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Last modification timestamp

4.2 Table: categories & products

categories

Column	Type	Notes
id	SERIAL	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
image_url	TEXT	Cloudinary image URL
image_public_id	VARCHAR(255)	Cloudinary public ID
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

products

Column	Type	Notes
id	SERIAL	PRIMARY KEY
category_id	INT	FK -> categories(id)
name	VARCHAR(255)	NOT NULL
price	DECIMAL(10,2)	NOT NULL
stock	INTEGER	DEFAULT 0
sku	VARCHAR(100)	UNIQUE

4.3 Tables: `cart` , `wishlist` , `orders` , `order_items` , `reviews`

`cart` (User Shopping Session)

Column	Type	Constraints
<code>id</code>	<code>SERIAL</code>	PRIMARY KEY
<code>user_id</code>	<code>INT</code>	FK -> users(id) ON DELETE CASCADE
<code>product_id</code>	<code>INT</code>	FK -> products(id) ON DELETE CASCADE
<code>quantity</code>	<code>INT</code>	DEFAULT 1

`wishlist` (Bookmarked Items)

Column	Type	Constraints
<code>id</code>	<code>SERIAL</code>	PRIMARY KEY
<code>user_id</code>	<code>INT</code>	FK -> users(id) ON DELETE CASCADE
<code>product_id</code>	<code>INT</code>	FK -> products(id) ON DELETE CASCADE

`orders` (Transactions)

Column	Type	Notes
<code>id</code>	<code>SERIAL</code>	PRIMARY KEY
<code>user_id</code>	<code>INT</code>	FK -> users(id) ON DELETE CASCADE
<code>total_price</code>	<code>DECIMAL(10,2)</code>	Final amount in USD
<code>status</code>	<code>VARCHAR(50)</code>	DEFAULT 'pending' (paid/shipped)
<code>shipping_address</code>	<code>TEXT</code>	Delivery address

`reviews` (User Ratings)

Column	Type	Notes
<code>id</code>	<code>SERIAL</code>	PRIMARY KEY
<code>product_id</code>	<code>INT</code>	FK -> products(id)
<code>user_id</code>	<code>INT</code>	FK -> users(id)
<code>rating</code>	<code>INTEGER</code>	Star rating (1 - 5)
<code>comment</code>	<code>TEXT</code>	Feedback text

4.4 JSONB Settings Specification (`settings`)

Admin configuration options are stored as key-value pairs utilizing PostgreSQL's `JSONB` type to avoid dynamic schema migrations when altering store rules.

Key	Category	JSONB Data Payload Structure
<code>store_info</code>	store	<code>{ "name": "My Furniture Shop", "email": "contact@shop.com", "phone": "+90 555 000 0000", "currency": "USD" }</code>
<code>stripe</code>	payment	<code>{ "enabled": true, "testMode": true }</code>
<code>shipping_rules</code>	shipping	<code>{ "flatRate": 10, "freeShippingOver": 100 }</code>
<code>cloudinary</code>	media	<code>{ "cloudName": "", "uploadPreset": "", "maxSizeMB": 5, "formats": ["jpg", "png", "webp"] }</code>

5. REST API Endpoint Specification

All REST API endpoints return standardized JSON responses. Protected endpoints require a valid JWT token passed via the `Authorization: Bearer <TOKEN>` header.

Endpoint Path	HTTP Method	Auth Role Required	Function & Purpose
/api/auth/signup	POST	Public	Registers new user account
/api/auth/login	POST	Public	Authenticates user & returns JWT
/api/auth/me	GET	Bearer Token	Returns authenticated user profile
/api/auth/profile	PUT	Bearer Token	Updates user info & Cloudinary avatar
/api/products	GET	Public	Fetches product list with filters
/api/products/:id	GET	Public	Fetches single product details
/api/products	POST	Admin Token	Creates new product + image upload
/api/categories	GET / POST	Public / Admin	Manages furniture categories
/api/cart	GET / POST / DELETE	Bearer Token	Manages user cart session & quantities
/api/wishlist	GET / POST / DELETE	Bearer Token	Manages user product bookmarks
/api/order/create-checkout-session	POST	Bearer Token	Generates Stripe payment checkout session
/api/order/webhook	POST	Stripe Signature	Stripe payment webhook clearance listener
/api/reviews/product/:id	GET / POST	Public / Bearer Token	Fetches and adds item reviews
/api/admin/stats	GET	Admin Token	Fetches store analytics & sales stats
/api/settings	GET / PUT	Admin Token	Reads and updates JSONB store settings

6. Security, Authentication & Webhooks

Security & Data Integrity Guarantees:

- Password Protection:** Hashed using bcrypt with 10 salt rounds prior to DB persistence.
- JWT Stateless Verification:** Tokens signed with secret keys, verified on protected routes.

- **Stripe Webhook Signature Verification: Raw body parsing ensures webhook payloads originate directly from Stripe.**
- **Rate Limiting & Helmet: HTTP security headers configured via `helmet` and DDoS rate limiting active on auth endpoints.**

7. Repository Structure & Operational Setup

7.1 Directory Organization

```
c:\Users\HP\furnitures\
├── backend/
│   ├── controllers/      # Route controllers (products, auth, orders, admin)
│   ├── database/         # PostgreSQL client setup & initDB script
│   ├── middleware/       # JWT auth verification, rate limiting, error handlers
│   ├── routes/           # Express REST API endpoint definitions
│   └── server.js         # Main Express API entry point & Stripe webhook listener
├── dist/                 # Compiled SSR & server bundle output
├── furniture-react/      # Frontend Application Code
│   └── src/
│       ├── client/       # Client browser entry (main.jsx)
│       └── shared/        # React components, pages, Redux slices, hooks
│           ├── components/ # UI modules (Header, Footer, ProductCard, AdminDashboard)
│           ├── contexts/   # React global state contexts
│           ├── services/   # Axios API client services
│           └── slices/     # Redux Toolkit state slices
├── ssr/                  # Webpack Server-Side Rendering template engine # PDF documentation generator
├── webpack.client.js     # Client React bundle Webpack configuration
└── webpack.server.js     # Server SSR bundle Webpack configuration
```

7.2 Environment Configuration (.env)

Variable Key	Description	Example / Default Value
PORT	Backend server port	5000
DATABASE_URL	PostgreSQL connection string	postgres://user:pass@localhost:5432/furniture
JWT_SECRET	Secret key for signing auth tokens	super_secret_jwt_key_12345
STRIPE_SECRET_KEY	Stripe API secret key	sk_test_...
STRIPE_WEBHOOK_SECRET	Stripe webhook signature secret	whsec_...
CLOUDINARY_CLOUD_NAME	Cloudinary account cloud name	demo_cloud

7.3 Quick Start & Build Instructions

Execution Commands:

1. Install Dependencies: `npm install`

2. Initialize Database: `node backend/database/initDB.js`

3. Run Development Server (Concurrent Client & Server): `npm run dev`